



# PROGRAMMING FUNDAMENTALS

WEEK 05 – IMPERATIVE PROGRAMMING

Course Instructor :  
Engr.Asma Khan

# PYTHON VARIABLES AND ASSIGNMENTS

- Functions are either called from within the interactive shell or by another program, which we will refer to as the calling program. In order to be able to design functions, we need to understand how values created in the calling program—or the interactive shell—are passed as input arguments to the function.
- To do this, however, we first need to understand exactly what happens in an assignment statement.
- Let's consider this question in the context of the assignment `a = 3`. First, let's note that before executing this assignment, the identifier `a` does not exist:

```
>>>a
```

```
>>> a
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    a
NameError: name 'a' is not defined
```

- In general, a Python assignment statement has this syntax:
- `<variable> = <expression>`

# AFTER EXECUTION THE INITIALIZATION OF VARIABLES

```
a = 3
b = 3.0
c = 'hello'
d = [2,3,5,8,11]
```

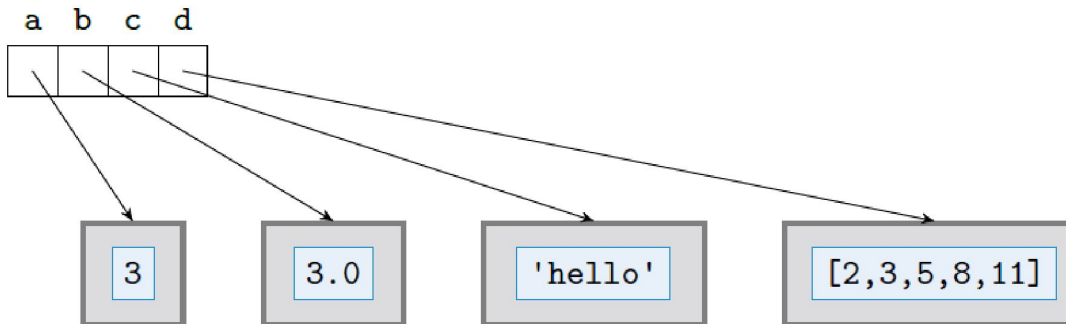
Python 3.6.7 Shell

File Edit Shell Debug Options Window Help

Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
RESTART: C:/Users/Faisal/Desktop/python Programming/variable_initialization.py
```

```
>>> a
3
>>> b
3.0
>>> c
'hello'
>>> d
[2, 3, 5, 8, 11]
```

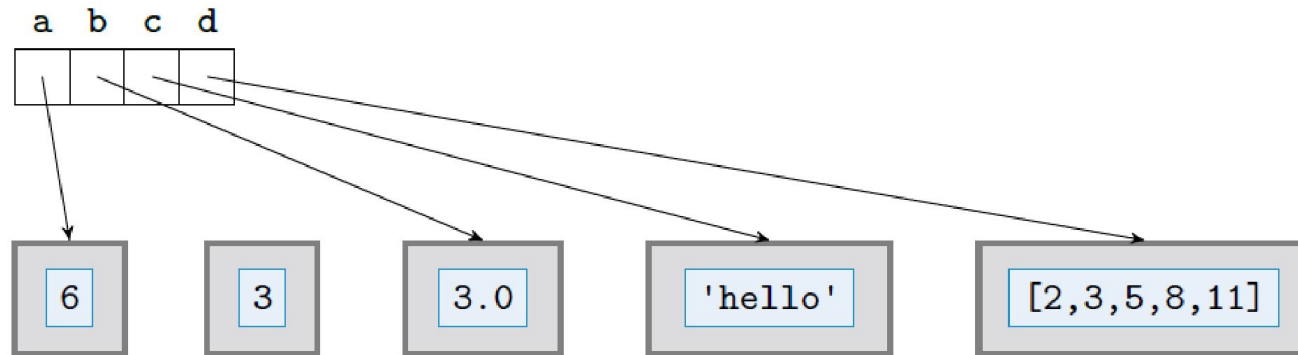


# MUTABLE AND IMMUTABLE TYPES

- Subsequent assignments to a, such as

```
>>> a = 6
```

- Will reuse the existing name a.
- The result of this assignment is that variable a will refer to another object, integer object 6. The int object 3 no longer is referred to by a variable, as shown in Figure



# MUTABLE AND IMMUTABLE TYPES

- The important thing to note is that the assignment `a = 6` did not change the value of the integer object 3.
- Instead, a new integer object 6 is created, and variable `a` now refers to it.
- In fact, there is no way to change the value of the object containing value 3.
- This illustrates an important feature of Python: Python int objects cannot be changed.
- Integer objects are not the only objects that cannot be modified.
- Types whose objects cannot be modified are called *immutable*.
- All Python number types (bool, int, float, and complex) are immutable.

# LISTS ARE MUTABLE

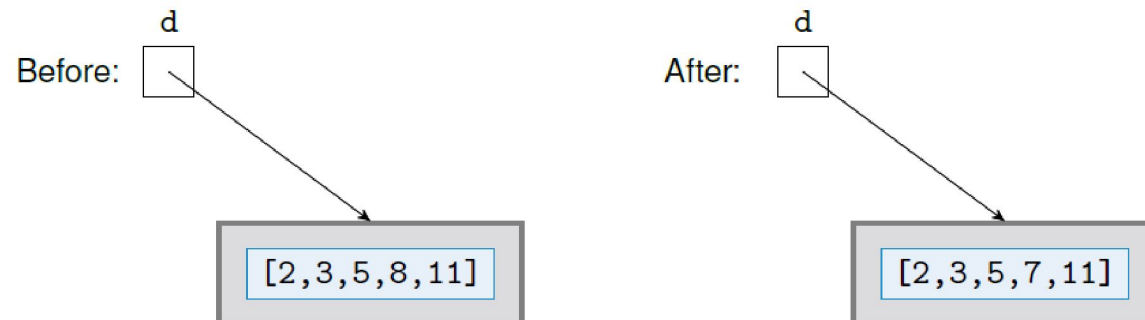
```
d = [2, 3, 5, 8, 11]
print(d)
d[3] = 7
print(d)
```

Python 3.6.7 Shell

File Edit Shell Debug Options Window Help

Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.

```
>>>
==== RESTART: C:/Users/Faisal/Desktop/python Programming/list_mutable.py ====
[2, 3, 5, 8, 11]
[2, 3, 5, 7, 11]
```



# WHAT ABOUT STRINGS?

 Python 3.6.7 Shell

File Edit Shell Debug Options Window Help

Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32

Type "help", "copyright", "credits" or "license()" for more information.

```
>>> c = 'hello'
```

```
>>> c[1] = 'i'
```

Traceback (most recent call last):

File "<pyshell#1>", line 1, in <module>

c[1] = 'i'

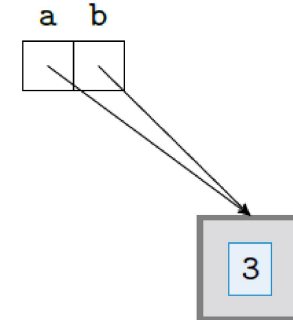
TypeError: 'str' object does not support item assignment

- We cannot modify a character of string object.
- The string type is *immutable*.

# ASSIGNMENTS AND MUTABILITY

- We often have the situation when multiple variables refer to the same object.
- (This is, in particular, the case when a value is passed as an input to a function.)
- We need to understand what happens when one of the variables is assigned another object. For example, suppose we do:  

```
>>> a = 3  
>>> b = a
```
- The first assignment creates an integer object with value 3 and gives it name a.
- In the second assignment, the expression a evaluates to the integer object 3, which then receives another name, b, as shown in Figure



# ASSIGNMENT AND MUTABILITY

- Variables a and b both refer to the same integer object 3. Now, what happens when we assign something else to a?
- `>>> a = 6`
- The assignment `a = 6` does not change the value of the object from 3 to 6 because the int type is **immutable**. Variable a should now refer to a new object with value 6.
- What about b?

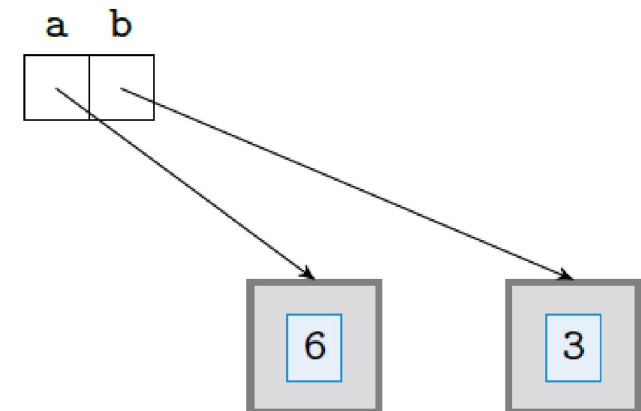
```
>>> a
```

```
6
```

```
>>> b
```

```
3
```

- Variable b still refers to the object with value 3, as shown in Figure



The point is this: If two variables refer to the same immutable object, that modifying one variable will not affect the other.

# MULTIPLE MUTABILITY

- Now let's consider what happens with lists. We start by assigning a list to `a` and then assigning `a` to `b`.

```
>>> a = [3, 4, 5]
```

```
>>> b = a
```

- We expect `a` and `b` to refer to the same list.
- That is indeed the case, as shown in Figure
- Now let's see what happens when we assign a new object to `b[1]`: `a` and vice versa.

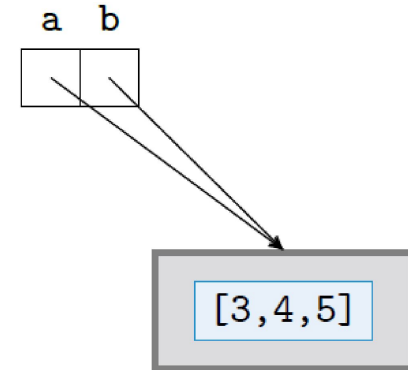
```
>>> b[1] = 8
```

```
>>> b
```

```
[3, 8, 5]
```

```
>>> a
```

```
[3, 8, 5]
```



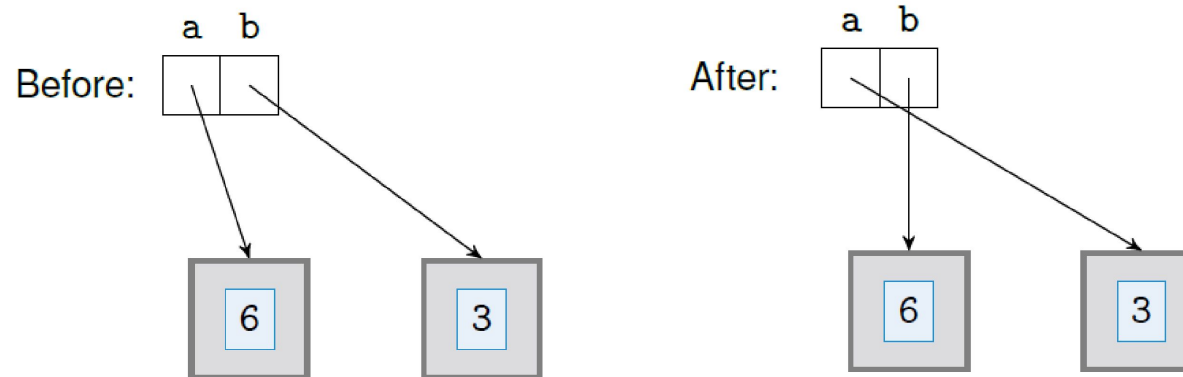
# SWAPPING

- We now consider a fundamental assignment problem. Let a and b refer to two distinct integer values:

```
>>> a = 6
```

```
>>> b = 3
```

- Suppose we need to swap the values of a and b. In other words, after the swap, a will refer to 3 and b will refer to 6, as shown in Figure



# SWAPPING (REGULAR METHOD)

- If we start by assigning the value of b to a:

`a = b`

- then variable a will refer to the same object that variable b refers to. So we will have both a and b refer to 3, and we would have “lost” integer object 6.
- Before we execute `a = b`, we must save a reference to 6 and then assign that to b at the end:

`>>> temp = a # temp refers to 6`

`>>> a = b # a refers to 3`

`>>> b = temp # b refers to 6`

# SWAPPING (THE PYTHON WAY)

- In Python, there is a much simpler way to achieve the swap. Python supports the multiple assignment statement:

```
>>> a = 6
```

```
>>> b = 3
```

```
>>> a, b = b, a
```

```
>>> a
```

```
3
```

```
>>> b
```

```
6
```

- In the multiple assignment statement `a, b = b, a`, the two expressions on the right of `=` are evaluated to two objects and then each is assigned to the corresponding variable.

# SWAPPING VALUES AND DATA TYPES AT A TIME

 Python 3.6.7 Shell

File Edit Shell Debug Options Window Help

Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.

```
>>> a = 'hello'
>>> b = 3.2
>>> c = (1,2,3,4)
>>> d = ['JAN', 'FEB', 'MAR']
>>> print(a,b,c,d)
hello 3.2 (1, 2, 3, 4) ['JAN', 'FEB', 'MAR']
>>> a, b, c, d = d, c, b, a
>>> print(a,b,c,d)
['JAN', 'FEB', 'MAR'] (1, 2, 3, 4) 3.2 hello
```

# PARAMETER PASSING

- we can understand how input arguments are passed in function calls.
- Functions are either called from within the
- interactive shell or by another program.
- We refer to either as the *calling program*.
- The input arguments in a function call are *names* of objects created in the calling program.
- These names may refer to objects that are mutable or immutable.
- We consider each case separately.

# IMMUTABLE PARAMETER PASSING

- We use the function `g()` to discuss the effect of passing a reference to an immutable object in a function call.

```
def g(x):  
    x = 5  
    return x
```

Python 3.6.7 Shell

File Edit Shell Debug Options Win

Python 3.6.7 (v3.6.7:6ec5cf24) on win32

Type "help", "copyright", "credits()"

>>>  
== RESTART: C:/Users/Faisal/Python367/Python367 Shell ==  
>>> g(2)  
5

```
def g(x):  
    print(x)  
    x = 5  
    return x
```

Python 3.6.7 Shell

File Edit Shell Debug Options Win

Python 3.6.7 (v3.6.7:6ec5cf24) on win32

Type "help", "copyright", "credits()"

>>>  
== RESTART: C:/Users/Faisal/Python367/Python367 Shell ==  
>>> g(2)  
5  
>>>  
== RESTART: C:/Users/Faisal/Python367/Python367 Shell ==  
>>> g(2)  
2  
5

```
def g(x):  
    x = a  
    print(x)  
    return x
```

Python 3.6.7 Shell

File Edit Shell Debug Options Win

Python 3.6.7 (v3.6.7:6ec5cf24) on win32

Type "help", "copyright", "credits()"

>>>  
== RESTART: C:/Users/Faisal/Python367/Python367 Shell ==  
>>> a = 15  
>>> g(a)  
15  
15

# IMMUTABLE PARAMETER PASSING

```
def g(x):  
    x = a  
    print(x)  
    return x
```

Python 3.6.7 Shell

File Edit Shell Debug Options Window Help

Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 4) on win32

Type "help", "copyright", "credits"

>>>

== RESTART: C:/Users/Faisal/Desktop/

>>> a = 10

>>> b = 6.5

>>> c = 'hello'

>>> d = (1, 3, 5, 7)

>>> e = ['Jan', 'Feb', 'Mar']

>>> g(a)

10

10

>>> g(b)

10

10

>>> g(c)

10

10

>>> g(d)

10

10

>>> g(e)

10

10

```
def g(x):  
    return x
```

>>> a = 10

>>> b = 5.6

>>> c = 'hello'

>>> d = (1, 3, 5, 7)

>>> e = ['Jan', 'Feb', 'Mar']

>>> g(a)

10

>>> g(b)

5.6

>>> g(c)

'hello'

>>> g(d)

(1, 3, 5, 7)

>>> g(e)

['Jan', 'Feb', 'Mar']

# MUTABLE PARAMETER PASSING

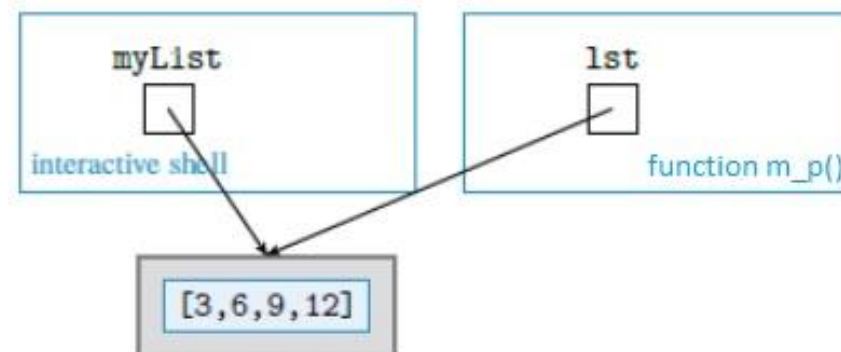
- In the assignment statement, a list object is created and assigned name myList.
- Then the function call h(myList) is made.
- When function m\_p() starts executing, the list referred to by myList will be assigned to variable name lst defined in the function definition of m\_p().
- So we have the situation illustrated in Figure.
- This example illustrates that when a mutable object, like list object [3,6,9,12], is passed as an argument in a function call, it may be modified by the function

mut\_p\_passing.py - C:/Users/Faisal/I

File Edit Format Run Options W

```
def m_p(lst):  
    lst[0] = 5  
    return lst
```

```
>>> myList = [3,6,9,12]  
>>> m_p(myList)  
[5, 6, 9, 12]
```





# PROGRAMMING PRACTICE

BASED ON UDF (USER DEFINED FUNCTIONS), CONDITIONS, LOOP

# PLUS FUNCTION()

- Define a function which can plus two parameters either integer or float.

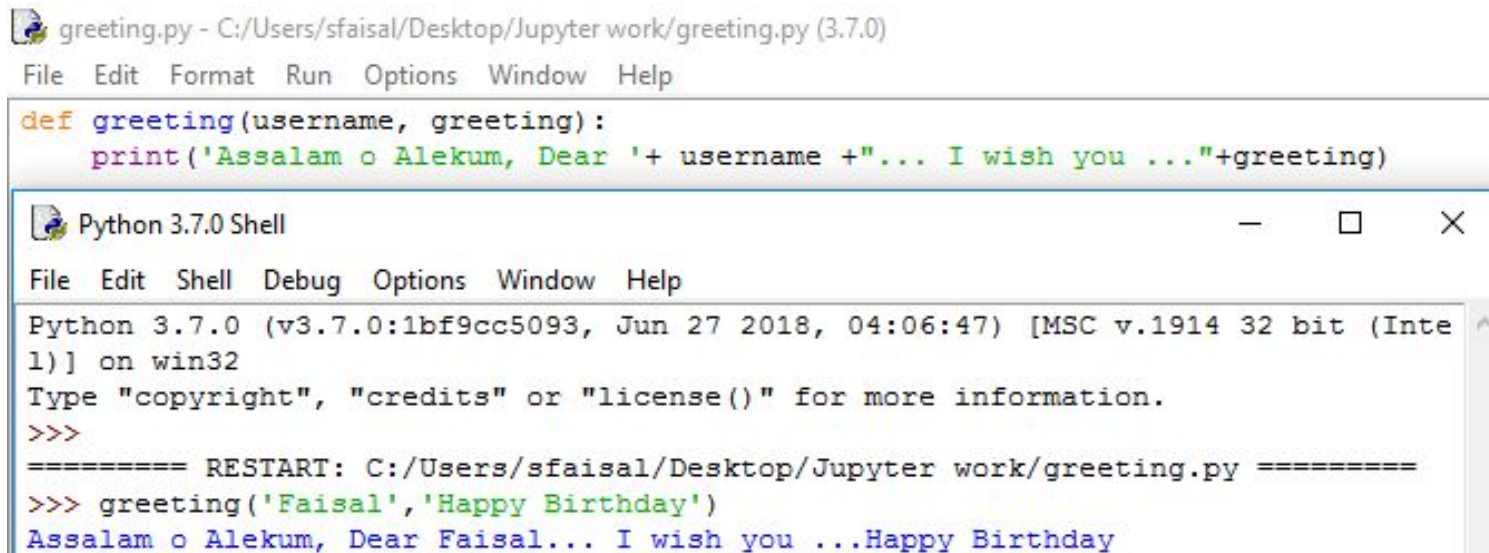
```
def plus(a,b):  
    print('You have entered the value of variable a :', a)  
    print('You have entered the value of variable b :', b)  
    print('Our Plus operator will generate :')  
    return a+b
```

 Python 3.7.0 Shell

```
File Edit Shell Debug Options Window Help  
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.  
1)] on win32  
Type "copyright", "credits" or "license()" for more information  
>>>  
===== RESTART: C:/Users/sfaisal/Desktop/Jupyter work/plus_f  
>>> plus(5,6)  
You have entered the value of variable a : 5  
You have entered the value of variable b : 6  
Our Plus operator will generate :  
11
```

# GREETING FUNCTION()

- Write a function which takes two inputs from the user as arguments one is user name and the other is greeting message.
- The function will then generate the message with user name and greetings.



The screenshot displays a Jupyter Notebook interface. The top window, titled 'greeting.py - C:/Users/sfaisal/Desktop/Jupyter work/greeting.py (3.7.0)', contains the following Python code:

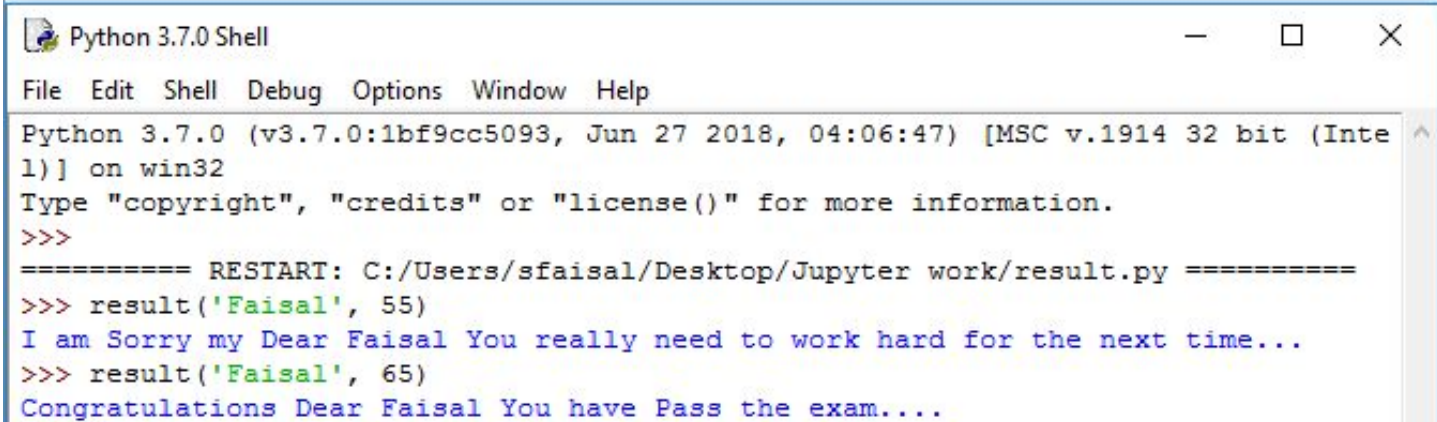
```
def greeting(username, greeting):  
    print('Assalam o Alekum, Dear ' + username + "... I wish you ..." + greeting)
```

The bottom window, titled 'Python 3.7.0 Shell', shows the execution of the code. It includes the standard Python 3.7.0 startup message and a restart notice. The function is then called with the arguments 'Faisal' and 'Happy Birthday', resulting in the output: 'Assalam o Alekum, Dear Faisal... I wish you ...Happy Birthday'.

# PASS/FAIL FUNCTION()

- Write a function which asks user to write his/her name, and the score in exam (1 to 100). If the marks are less than 60 it will reply a sorry message, otherwise it will congratulate the user.

```
def result(name, marks):  
    if marks>60:  
        print("Congratulations Dear " + name + " You have Pass the exam....")  
    else:  
        print("I am Sorry my Dear " + name + " You really need to work hard for the next time...")
```



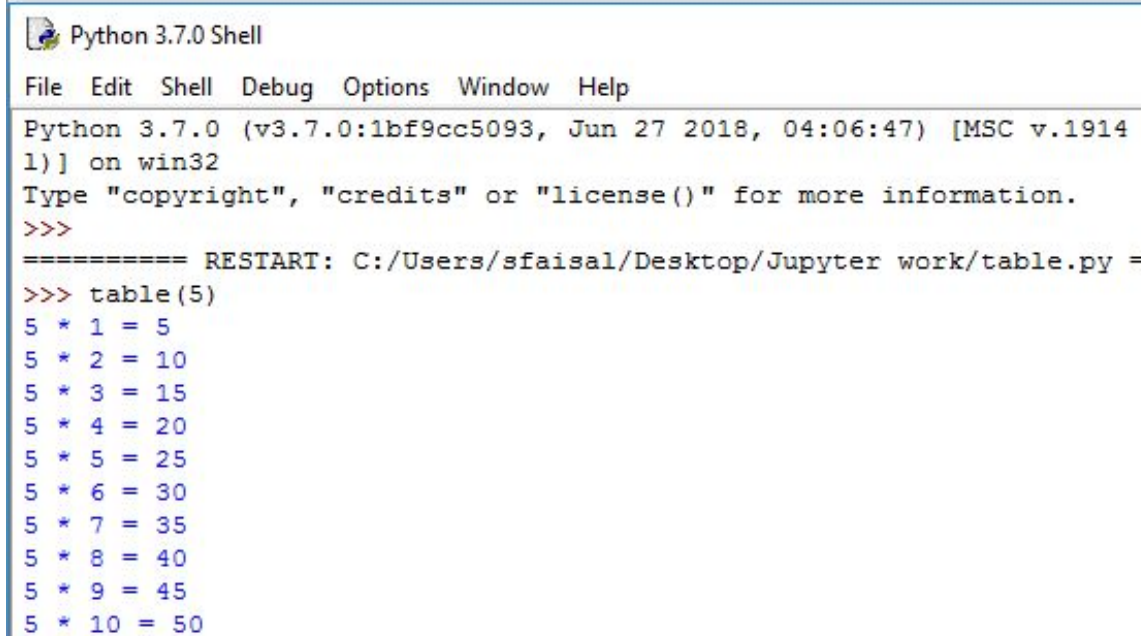
The screenshot shows a Python 3.7.0 Shell window with a menu bar (File, Edit, Shell, Debug, Options, Window, Help). The command prompt displays the Python version and architecture. After a restart, the user enters two commands: `result('Faisal', 55)` and `result('Faisal', 65)`. The first command outputs "I am Sorry my Dear Faisal You really need to work hard for the next time..." and the second outputs "Congratulations Dear Faisal You have Pass the exam....".

```
Python 3.7.0 Shell  
File Edit Shell Debug Options Window Help  
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32  
Type "copyright", "credits" or "license()" for more information.  
>>>  
===== RESTART: C:/Users/sfaisal/Desktop/Jupyter work/result.py =====  
>>> result('Faisal', 55)  
I am Sorry my Dear Faisal You really need to work hard for the next time...  
>>> result('Faisal', 65)  
Congratulations Dear Faisal You have Pass the exam....
```

# TABLE FUNCTION()

- Write a function which will generate a table starting from 1 till 10. The function will take one parameter as integer or float and generate the table such as  $4 * 1 = 4$ .

```
def table(number):  
    seq = [1,2,3,4,5,6,7,8,9,10]  
    for n in seq:  
        print(str(number) + " * " + str(n) + " = " + str(number * n))
```



The screenshot shows a Python 3.7.0 Shell window with a menu bar (File, Edit, Shell, Debug, Options, Window, Help). The command prompt shows the execution of the `table(5)` function, which prints a multiplication table for the number 5 from 1 to 10.

```
Python 3.7.0 Shell  
File Edit Shell Debug Options Window Help  
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914  
1)] on win32  
Type "copyright", "credits" or "license()" for more information.  
>>>  
===== RESTART: C:/Users/sfaisal/Desktop/Jupyter work/table.py =  
>>> table(5)  
5 * 1 = 5  
5 * 2 = 10  
5 * 3 = 15  
5 * 4 = 20  
5 * 5 = 25  
5 * 6 = 30  
5 * 7 = 35  
5 * 8 = 40  
5 * 9 = 45  
5 * 10 = 50
```



# ASSIGNMENT # 3

TO BE SUBMITTED BY SOLVING IT ON PYTHON IDLE & EMAIL IT TO ME. REMEMBER TO READ CHAPTER 3

## PRACTICAL PROBLEM 3.14

- Draw a diagram representing the state of names and objects after this execution:

```
>>> a = [5, 6, 7]
```

```
>>> b = a
```

```
>>> a = 3
```

## PRACTICAL PROBLEM 3.15

- Suppose a nonempty list `team` has been assigned. Write a Python statement or statements that swap the first and last value of the list. So, if the original list is:

```
>>> team = ['Ava', 'Eleanor', 'Clare', 'Sarah']
```

- then the resulting list should be:

```
>>> team
```

```
['Sarah', 'Eleanor', 'Clare', 'Ava']
```

# READING ASSIGNMENT

- Read chapter 3 complete. In case of any problem please come to my office.
- Consultation / Tutorial hours are available for your help.



# END OF LECTURE – WEEK 5